

제1강

1) 자바스크립트 표현식

- JSX에서 자바스크립트 표현식을 구성하는 변수, 연산자(AND, OR, 삼항, 스프레드) 사용방법을 알아본다.

[src]-[components01] Expression.jsx

```
import '../Style01.css'

const Expression = () => {
  const name='Chris';
  const age = 20;
  const job = 'Student';
  const isLogin=true;
  const student = { name, age }
  const person = { ...student, job }

  return (
    <div className='box'>
      <h1>변수값 출력</h1>
      <h3>{name || 'Justin'}, {age+10}, {job || 'Programer'}, {isLogin.toString()}</h3>
      <h3>{Name:${name}, Age:${age}, Job:${job}}</h3>

      <h1>삼항 연산자</h1>
      <h3>{isLogin ? <button>Logout</button>:<button>Login</button>}</h3>

      <h1>AND 연산자</h1>
      <h3>{isLogin && <div>{name || 'Justin'}, {age || 0}, {job || 'Programer'}</div>}</h3>
      <h3>{!isLogin && <div>Please Login!</div>}</h3>

      <h1>OR 연산자</h1>
      <h3>{age || 0}</h3>
      <h3>{job || 'Programer'}</h3>

      <h1>스프레드 연산자</h1>
      <h3>{person.name || 'Justin'}, {person.age || 0}, {person.job || 'Programer'}</h3>
    </div>
  )
}
export default Expression
```

2) 스타일 적용

- JSX 요소에 직접 스타일을 적용하는 인라인 방식, className으로 스타일을 적용해 본다.

[src]-[components01] StylePage.jsx

```
import '../Style01.css'

const StylePage = () => {
  const style = {
    fontSize:'30px',
    fontFamily:'돋움체',
    color:'red',
  }

  return (
    <div className='box'>
      <h1 className='hello'>안녕 리엑트!</h1>
      <h2 style={{fontSize:'50px',fontFamily:'궁서체',color:'blue'}}>저는 홍길동입니다.</h2>
      <h3 style={style}>만나서 반갑습니다.</h3>
    </div>
  )
}
export default StylePage
```

```
.hello {
  background-color: black;
  font-size: 50px;
  color:aqua;
}
```

3) 컴포넌트 생성

- MainPage 컴포넌트에 Header, Content, Footer 컴포넌트를 생성하여 등록한다. 생성 시 RAECA 단축키를 사용한다.
- MenuBar 컴포넌트를 생성하여 Header, Footer 컴포넌트에 등록한다. 컴포넌트는 원하는 곳 어디든지 재사용이 가능하다.

[src]-[components01] MainPage.jsx

```
import '../Style01.css'
import Header from './Header'
import Content from './Content'
import Footer from './Footer'
const MainPage = () => {
  return (
    <div>
      <h1>홈페이지</h1>
      <Header/>
      <Content/>
      <Footer/>
    </div>
  )
}
export default MainPage
```

[src]-[components01] Header.jsx

```
import MenuBar from './MenuBar'
const Header = () => {
  return (
    <div>
      <img src='http://picsum.photos/960/200'/>
      <MenuBar/>
    </div>
  )
}
export default Header
```

[src]-[components01] Content.jsx

```
const Content = () => {
  return (
    <div className='content'>
      <h1>Welcome to My Homepage!</h1>
    </div>
  )
}
export default Content
```

```
.content {
  height: 500px;
  display: flex;
  align-items:center;
  justify-content:center;
}
```

[src]-[components01] Footer.jsx

```
const Footer = () => {
  return (
    <div>
      <hr/>
      <h3>Copyright 2025. 홍길동 All rights reserved.</h3>
    </div>
  )
}
export default Footer
```

[src]-[components01] MenuBar.jsx

```
const MenuBar = () => {
  return (
    <div className='menu'>
      <a href='#'>네이버</a>
      <a href='#'>다음</a>
      <a href='#'>구글</a>
    </div>
  )
}
export default MenuBar
```

```
.menu {
  font-size: 30px;
  margin: 20px 0px;
}

a {
  padding-right: 50px;
}
```

제2강

1) props를 사용한 학생 목록

- 부모컴포넌트에서 학번, 성명, 학과를 props로 설정하여 자식컴포넌트로 전달하여 학생정보를 출력한다.

[src]-[components02] StudentPage.jsx

```
import './Style02.css'
import Student from './Student'

const students = [
  { id: 1, name: '홍길동', dept: '컴정과' },
  { id: 2, name: '심청이', dept: '전자과' },
  { id: 3, name: '이순신', dept: '건축과' },
]

const StudentPage = () => {
  return (
    <div className='box'>
      <h1>학생목록</h1>
      {students.map(s=>
        <Student key={s.id} student={s}/>
      )}
    </div>
  )
}
```

export default StudentPage

- Student 컴포넌트를 감싸주는 <div> 태그를 < 프래그먼트 태그로 변경하면 box 클래스아래 h3가 자식이 되어 스타일이 적용된다.

[src]-[componetns02] Student.jsx

```
const Student = ({student}) => {
  const {id, name, dept} = student;

  return (
    <div>
      <h3>학번:{id}</h3>
      <h3>이름:{name}</h3>
      <h3>학과:{dept}</h3>
      <br/>
    </div>
  )
}
```

export default Student

[src]-[componetns02] Student.jsx

```
const Student = ({student}) => {
  const {id, name, dept, age}= student

  return (
    <>
      <h3>학번:{id}</h3>
      <h3>이름:{name}</h3>
      <h3>학과:{dept}</h3>
      <h3>나이:{age}</h3>
      <br/>
    </>
  )
}
```

export default Student

```
.box > h3 {
  display: inline;
  margin-right: 20px;
}
```

2) props를 사용한 상품 목록

- 부모컴포넌트에서 상품 번호, 이름, 가격을 props로 설정하여 자식컴포넌트로 전달하여 상품정보를 출력한다.
- <table> 아래에는 <tr>만 사용할 수 있다. <tr>를 여러 개 사용할 경우 프래그먼트를 사용하여 결합하고 key 지정을 위해 <React.Fragment>를 사용한다.

[src]-[componetns02] ProductPage.jsx

```
import React from 'react'
import Product from './Product'
import './Style02.css'

const products = [
  { id:1,name:'삼성 세탁기',price:2500000,description:'다채로운 색상 조합은 어느 공간에서나 조화롭게 어우러짐'},
  { id:2,name:'엘지 냉장고',price:3500000,description:'AI 하이브리드 쿨링과 키친핏 디자인, 스마트싱스 연동 등 개인화 및 편의성을 극대화'},
  { id:3,name:'삼성 스타일러',price:150000,description:'옷을 위아래로 털어주며 좌우 흔들림(트위스트)을 더해 먼지를 효과적으로 제거' },
]

const ProductPage = () => {
  return (
    <div className='box'>
      <h1>상품목록</h1>
      <table>
        <tbody>
          {products.map(p=>
            <React.Fragment key={p.id}>
              <Product product={p}/>
              <tr style={{textAlign:'left'}}>
                <td colSpan={3}>{p.description}</td>
              </tr>
            </React.Fragment>
          )}
        </tbody>
      </table>
    </div>
  )
}
export default ProductPage
```

[src]-[componetns02] Product.jsx

```
const Product = ({product}) => {
  const { id, name, price } = product;
  return (
    <tr>
      <td>{id}</td>
      <td>{name}</td>
      <td>{parseInt(price).toLocaleString()}</td>
    </tr>
  )
}
export default Product
```

[src]-[componetns02] Product.jsx

```
const Product = ({product}) => {
  const { id, name, price } = product;
  return (
    <>
      <tr>...</tr>
      <tr style={{background:'gray', color:'white'}}>
        <td colSpan={3}>상품설명</td>
      </tr>
    </>
  )
}
export default Product
```

3) state를 사용한 카운터 증감

- useState Hook 함수를 이용하여 number를 state 변수로 정의한 후 number를 증감한다.

[src]-[componetns02] CounterPage.jsx

```
import React, { useState } from 'react'
import './Style02.css'

const CounterPage = () => {
  //let number = 0;
  const [number, setNumber] = useState(0);

  const onIncrease = () => {
    //number++;
    console.log(number);
    setNumber(number+1);
  }
  const onDecrease = () => {
    //number--;
    console.log(number);
    setNumber(number-1);
  }

  return (
    <div className='box'>
      <h1>{number}</h1>
      <button onClick={onDecrease}>감소</button>
      <button onClick={onIncrease}>증가</button>
    </div>
  )
}
```

4) state를 사용한 인사말과 색상 변경

- message, color를 useState Hook 함수를 사용하여 state 변수로 정의 한 후 버튼 클릭한 경우 인사말과 색상을 변경한다.

[src]-[componetns02] SayPage.jsx

```
import React, { useState } from 'react'
import './Style02.css'

const SayPage = () => {
  const initMessage = 'Please press the button!'
  const [message, setMessage] = useState(initMessage);
  const [color, setColor] = useState('black');

  return (
    <div className='box'>
      <h1 style={{color:color}}>{message}</h1>
      <div>
        <button onClick={()=>setMessage('Hello!')}>입장</button>
        <button onClick={()=>setMessage('Good bye!')}>퇴장</button>
      </div>
      <div>
        <button onClick={()=>setColor('red')}>빨강색</button>
        <button onClick={()=>setColor('green')}>초록색</button>
        <button onClick={()=>setColor('blue')}>파랑색</button>
      </div>
    </div>
  )
}
```

제3강

1) onClick, onContextMenu 핸들러

- 증감 버튼에서 마우스 왼쪽을 클릭하면 number가 1증가하고 증감 버튼에서 마우스 오른쪽을 클릭하면 number가 1증가한다.
- ON/OFF 버튼을 클릭할 때마다 버튼의 캡션이 Toggle되고 OFF 버튼이면 증감 버튼이 클릭되지 못하도록 한다.

[src]-[componetns03] CounterPage.jsx

```
...
const CounterPage = () => {
  const [number, setNumber] = useState(0);

  const onClickLeft = () => {
    setNumber(number+1);
  }
  const onClickRight = (e) => {
    e.preventDefault();
    setNumber(number-1);
  }
  const onKeyDown = (e) => {
    if(e.key==='Enter') alert('현재 값은 ${number}입니다.');
```

2) onChange, onKeyDown 핸들러

- 입력상자에서 키를 입력하면 state 변수 값이 변경되고 엔터키를 누르면 경고 창을 띄운다.
- useRef() 함수 Hook을 사용하여 특정 DOM 요소에 직접 접근해 포커스(focus)를 지정한다.

[src]-[componetns03] InfoPage.jsx

```
...
const InfoPage = () => {
  const [name, setName] = useState('');
  const [age, setAge] = useState(0);
  const nameRef = useRef(null);

  const onClick = () => {
    alert('Name:${name}, Age:${age}');
    nameRef.current.focus();
  }
  const onKeyDown = (e) => {
    if(e.key==='Enter') onClick();
  }
  return (
    <div className='box'>
      <h3>이름:{name || '무기명'} | 나이:{age}</h3>
      <input ref={nameRef} value={name} onChange={(e)=>setName(e.target.value)} onKeyDown={onKeyDown}
        placeholder='이름'/><br/>
      <input value={age} onChange={(e)=>setAge(e.target.value)} onKeyDown={onKeyDown}
        placeholder='나이' type='number' setp={1}/><br/>
      <button onClick={onClick}>확인</button>
    </div>
  )
}
```

3) OnSubmit, OnReset 핸들러

- 상품목록을 출력하고 입력상자 내용이 변경될 때마다 form값 변경 onSubmit에 등록 함수, onReset에 초기화 함수를 등록 한다.
- 상품을 등록할 경우 useRef를 이용해 id 증가시키고 이름 입력상자에 포커스 지정한다.
- 상품 목록에서 특정 행에서 마우스 오른쪽 버튼을 클릭한 경우 해당상품을 삭제 한다.

[src]-[componetns03] RegisterPage.jsx

```
import React, { useRef, useState } from 'react'
import './Style03.css'

const data = [ {id:1, name:'삼성 세탁기', price: 2500000}, { id:2, name:'엘지 냉장고', price:3500000}];
const initForm = {name:'엘지 TV', price:2000000};

const RegisterPage = () => {
  const [products, setProducts] = useState(data);
  const idRef = useRef(4);
  const nameRef = useRef(null);

  const [form, setForm] = useState(initForm);
  const {name, price} = form;
  const onChange = (e) => {
    setForm({...form, [e.target.name]:e.target.value});
  }

  const onSubmit = (e) => {
    e.preventDefault();
    setProducts([...products, { id:idRef.current++, ...form }]);
    nameRef.current.focus();
  }

  const onReset = (e) => {
    e.preventDefault();
    setForm(initForm);
    nameRef.current.focus();
  }

  const onRemove = (e, id) => {
    e.preventDefault();
    if(window.confirm(`${id}번 상품을 삭제하실래요?`)){
      const data = products.filter(p=>p.id !== id);
      setProducts(data);
    }
  }

  return (
    <div className='box'>
      <h1>상품등록</h1>
      <form onSubmit={onSubmit} onReset={onReset}>
        <input value={name} onChange={onChange} ref={nameRef}
          placeholder='상품명' name='name' />
        <input value={price} onChange={onChange}
          placeholder='상품가격' name='price' type='number' step={1000} />
        <div>
          <button type='submit'>등록</button>
          <button type='reset'>취소</button>
        </div>
      </form>
      <table>
        <tbody>
          {products.map(p=>
            <tr key={p.id} onContextMenu={(e)=>onRemove(e, p.id)}>
              <td>{p.id}</td>
              <td>{p.name}</td>
              <td>{parseInt(p.price).toLocaleString()}원</td>
            </tr>
          )}
        </tbody>
      </table>
    </div>
  )
}

export default RegisterPage
```

제4강

1) useEffect() 함수

- useEffect() 함수는 리액트 컴포넌트가 렌더링 될 때마다 특정 작업을 실행할 수 있도록 하는 Hook이다.
- age state 변수가 변경될 때마다 특정 작업을 실행하고 싶으면 useEffect() 함수 두 번째 인자 의존 배열에 age를 넣어준다.

[src]-[components04] InfoPage.jsx

```
const InforPage = () => {  
  ...  
  useEffect(()=>{  
    console.log(name, age);  
  }, [age]);  
}
```

2) useEffect() 함수를 이용한 Posts 데이터 출력

- useEffect() 함수를 이용해 page가 변경될 때마다 jsonplaceholder 사이트 posts 5개의 데이터를 불러오는 API 함수를 호출한다.
- posts의 제목(title)을 클릭할 때마다 본문(body)가 보였다, 사라졌다 Toggle되는 기능을 추가한다.

[src]-[components04] PostPage.jsx

```
...  
const PostPage = () => {  
  const [posts, setPosts] = useState([]);  
  const [page, setPage] = useState(1);  
  const size = 5;  
  const lastRef = useRef(1);  
  
  const callAPI = () => {  
    fetch('https://jsonplaceholder.typicode.com/posts')  
    .then(response => response.json())  
    .then(json => {  
      const start = (page-1) * size + 1;  
      const end = page * size;  
      const data = json.filter(post=>post.id>=start && post.id<=end);  
      console.log(data);  
      setPosts(data);  
      lastRef.current = Math.ceil(json.length/size);  
    });  
  }  
  
  useEffect(()=>{  
    callAPI();  
  }, [page]);  
  
  const onClick = (id) => {  
    const data = posts.map(post=>post.id===id ? { ...post, isVisible:!post.isVisible } : post);  
    setPosts(data);  
  }  
  
  return (  
    <div className='box'>  
      <h1>Posts</h1>  
      {posts.map(post=>  
        <div key={post.id}>  
          <h5 onClick={()=>onClick(post.id)} className='title'>{post.id}. {post.title}</h5>  
          { post.isVisible && <div className='body'>{post.body}</div>  
        </div>  
      )}  
      <PageButton page={page} last={lastRef.current} setPage={setPage}/>  
    </div>  
  )  
}  
export default PostPage
```

3) useEffect() 함수를 이용한 Todos 데이터 출력

- useEffect() 함수를 이용해 page가 변경될 때마다 jsonplaceholder 사이트 todox 5개의 데이터를 불러오는 API 함수를 호출한다.
- todos의 completed가 바뀔 때마다 체크상태 Toggle하고 선택, 해지할 때마다 전체 completed 체크상태 변경한다.

[src]-[components04] TodoPage.jsx

```
import React, { useState, useEffect, useRef } from 'react'
import './Style04.css'
import PageButton from './PageButton';

const TodoPage = () => {
  const [page, setPage]= useState(1);
  const [todos, setTodos] = useState([]);
  const [checked, setChecked] = useState([]);
  const lastRef = useRef(1);
  const size=5;

  const callAPI = () => {
    fetch('https://jsonplaceholder.typicode.com/todos')
    .then(response => response.json())
    .then(json => {
      //console.log(json);
      const start = (page-1) * size + 1;
      const end = (page * size)
      const data = json.filter(todo=>todo.id>=start && todo.id<=end);
      console.log(data)
      setTodos(data);
      lastRef.current=Math.ceil(json.length/size);
    });
  }

  useEffect(()=>{
    callAPI();
  }, [page]);

  useEffect(()=>{
    setChecked(todos.filter(todo=>todo.completed));
  }, [todos]);

  const onChange = (e, id) => {
    const data = todos.map(todo=>todo.id===id ? { ...todo, completed:e.target.checked } : todo);
    setTodos(data);
  }

  const onChangeAll = (e) => {
    const data = todos.map(todo=>({ ...todo, completed:e.target.checked }));
    setTodos(data);
  }

  return (
    <div className='box'>
      <h1>Todos</h1>
      <input type='checkbox' onChange={ onChangeAll } checked={ todos.length===checked.length }/>
      <span>전체선택/해지</span>
      <hr/>
      { todos.map(todo=>
        <div key={todo.id}>
          <input type='checkbox' onChange={(e)=>onChange(e, todo.id)} checked={ todo.completed }/>
          <span className='title'>{todo.title}</span>
        </div>
      )}
      <PageButton page={page} setPage={setPage} last={lastRef.current}/>
    </div>
  )
}

export default TodoPage
```